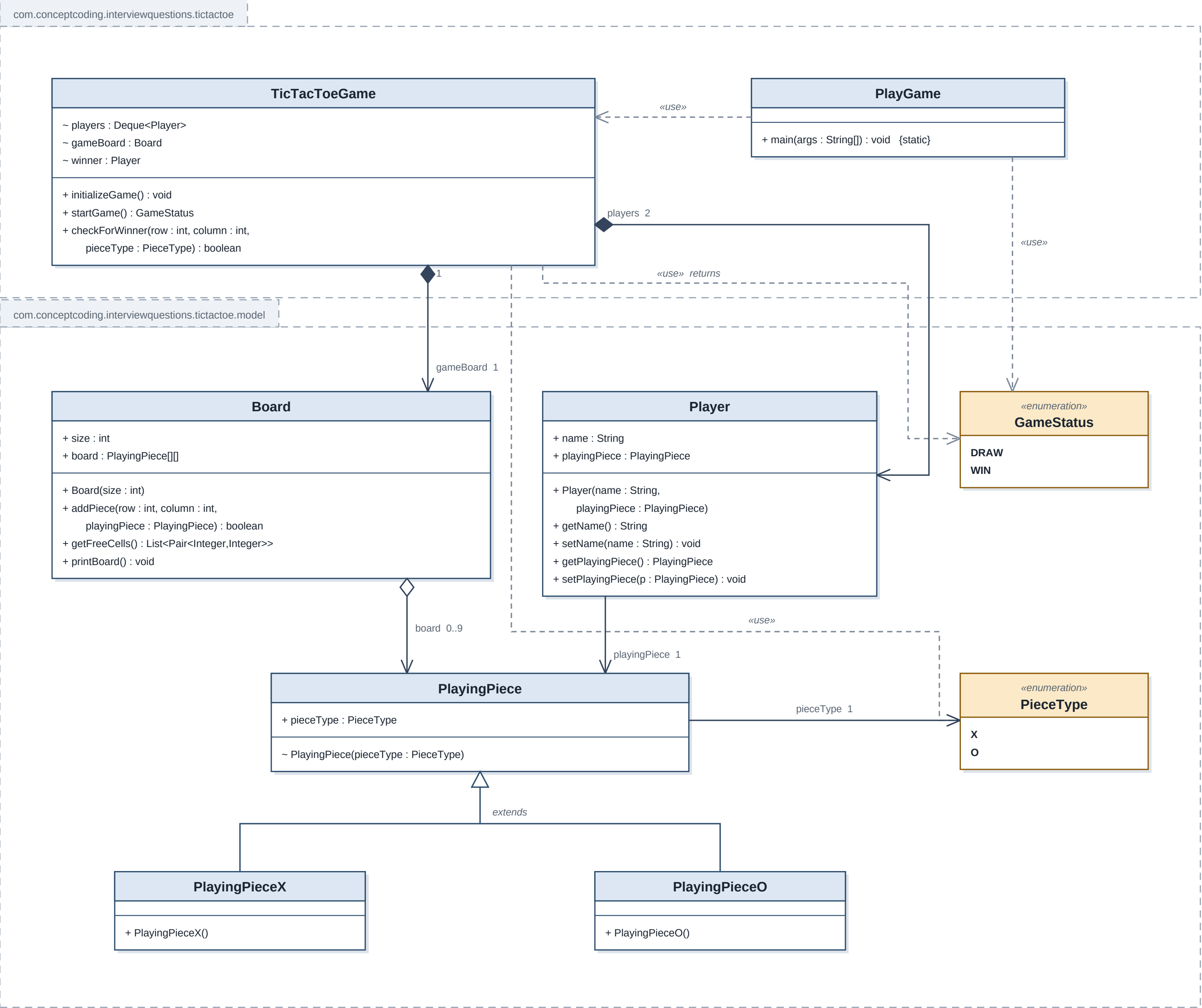


Tic Tac Toe — UML Class Diagram

9 types across 2 packages · visibility: + public ~ package-private · reverse-engineered from the Java source



Legend

- Association
"knows / refers to"
- Composition
whole creates & owns the part
- Aggregation
whole holds parts it may outlive
- Dependency
uses as parameter / return type
- Generalization
extends (inheritance)
- Package
dashed frame = Java package
- Enum type
amber header = Java enum

Design notes

- TicTacToeGame is the controller: it creates both players and the board, then drives the turn loop. The Deque<Player> is used as a round-robin queue — removeFirst() takes the turn, addLast() sends the player to the back (addFirst() on an invalid move so the turn is retried).
- winner points at one of the two Player objects already held in players, so it is a plain reference, not a second ownership relationship.
- Board is a passive grid: it stores PlayingPiece[][] and validates that a cell is empty, but knows no win rules — checkForWinner() lives in TicTacToeGame.
- PlayingPieceX / PlayingPieceO add no state or behaviour; they only pin the PieceType in the constructor. Because PlayingPiece's constructor is package-private (~), pieces can only be created inside the model package.
- Board.getFreeCells() returns List<Pair<Integer,Integer>> from organtlr.v4.runtime.misc.Pair — an external ANTLR type leaking into the domain model. A small Cell/Position record would remove that third-party dependency.
- Fields on Board, Player and PlayingPiece are public, so the getters/setters on Player are redundant. Making them private would tighten encapsulation.
- checkForWinner() re-tests both diagonals on every move, even when the move is not on one, and startGame() mixes console I/O (Scanner, printBoard) with game logic — both are natural extraction points if the game grows.
- GameStatus only models WIN and DRAW, so startGame() can never return anything else and the default branch of the switch in PlayGame is unreachable today. An IN_PROGRESS / ABANDONED value would make the enum honest about the loop's real states.
- The model already generalises beyond 3×3: Board takes size in its constructor and checkForWinner() loops to gameBoard.size, so an n×n variant needs no structural change — only input validation against size.